

Evaluador de expresiones con números duales

Luciano David Duarte

Análisis de Lenguajes de Programación
Licenciatura en Ciencias de la Computación – UNR

Abstract

Este trabajo implementa un lenguaje pequeño para escribir expresiones matemáticas y evaluarlas en un punto. Además de calcular el valor de la expresión, el sistema calcula su derivada usando números duales, una técnica de diferenciación automática. El proyecto comenzó como un DSL/EDSL centrado en números duales y luego se extendió hacia una calculadora de expresiones, con parser, procesamiento de archivos, manejo explícito de errores, optimizaciones conservadoras y una mónada propia de evaluación.

1 Introducción

El objetivo principal del proyecto es mostrar cómo representar y evaluar expresiones matemáticas en Haskell. La idea inicial fue construir una herramienta acotada para expresar funciones en una variable y observar el comportamiento de los números duales. Con el avance del trabajo se agregaron funcionalidades de calculadora, pero el eje conceptual sigue siendo la diferenciación automática.

El proyecto no busca ser un sistema de álgebra computacional completo. En particular, no implementa derivación simbólica general ni integración. La decisión fue mantener un alcance razonable para la materia y concentrar el esfuerzo en robustez, claridad del AST, evaluación segura y conexión con los contenidos de mónadas.

2 Objetivos

Los objetivos principales son:

- representar expresiones matemáticas mediante un tipo algebraico;
- parsear expresiones desde texto;
- evaluar expresiones en un valor de la variable principal;
- calcular derivadas usando números duales;
- manejar errores de dominio sin depender de excepciones parciales;
- incorporar una mónada propia vinculada con los temas de la cátedra.

Como objetivos secundarios se buscó ordenar el código para futuras extensiones, documentar las decisiones de diseño y dejar una base parcial para múltiples variables a nivel de API.

3 Representación de expresiones

Las expresiones se representan con el tipo `Expr`. Es un tipo algebraico que incluye literales, variables, operadores binarios y funciones unarias.

```
data Expr
  = Lit Double
  | Var String
  | Add Expr Expr
  | Sub Expr Expr
  | Mul Expr Expr
  | Div Expr Expr
  | Pow Expr Expr
  | Sin Expr
  | Cos Expr
  | Log Expr
  | Exp Expr
  | Sqrt Expr
```

Esta representación permite recorrer el árbol por pattern matching. Esa es una ventaja importante de Haskell para implementar un DSL de este estilo, porque las reglas de evaluación quedan escritas directamente sobre la estructura sintáctica.

4 Parser

El parser se implementa con Parsec. Se definió un lexer para reconocer operadores, literales, identificadores, constantes y funciones reservadas. La precedencia usada es:

potencia > menos unario > multiplicación/división > suma/resta

Una mejora importante fue exigir que el parser consuma toda la entrada. Por ejemplo, una cadena como:

`x + 1 basura`

se rechaza en vez de aceptarse parcialmente. Esto evita resultados inesperados y vuelve el lenguaje más estricto.

También se corrigió la interpretación de $-x^2$. Actualmente se parsea como:

$$-(x^2)$$

lo cual coincide con la convención matemática usual.

5 Números duales

Un número dual tiene la forma:

$$a + b\varepsilon$$

donde:

$$\varepsilon^2 = 0$$

Si se evalúa una función diferenciable en $x + \varepsilon$, se obtiene:

$$f(x + \varepsilon) = f(x) + f'(x)\varepsilon$$

En el código se representa con:

```
data Dual = Dual
  { primal :: Double
  , deriv  :: Double
  }
```

La parte **primal** almacena el valor de la función y la parte **deriv** almacena la derivada. Por ejemplo, para derivar respecto de x , se evalúa la expresión usando el dual `Dual x 1`.

6 Evaluación escalar y evaluación dual

El sistema ofrece dos caminos de evaluación:

```
eval :: Expr -> Double -> Either ErrorType Double
evalDual :: Expr -> Double -> Either ErrorType Dual
```

`eval` calcula únicamente el valor numérico. `evalDual` calcula valor y derivada en una sola pasada usando números duales.

Además se agregaron versiones con entorno:

```
evalWithEnv :: EvalEnv Double -> Expr -> Either ErrorType Double
evalDualWithEnv :: EvalEnv Dual -> Expr -> Either ErrorType Dual
```

Esto permite evaluar expresiones con más variables desde la API. El modo interactivo y el modo archivo siguen centrados en la variable `x`, pero el diseño ya no queda completamente atado a una única variable.

7 Mónada de evaluación

Para ordenar el manejo de errores y de variables se incorporó una mónada propia:

```
newtype EvalM env a =
  EvalM (EvalEnv env -> Either ErrorType a)
```

Conceptualmente, una computación `EvalM env a`:

- lee un entorno de variables;
- puede fallar con un `ErrorType`;
- si no falla, produce un valor de tipo `a`.

En términos de la materia, combina una idea similar a `Reader` con una idea similar a `Either`. No se usó un transformador de mónadas porque para el alcance del trabajo era más claro implementar el tipo propio y mostrar directamente sus instancias.

La función que ejecuta la computación es:

```
runEvalM :: EvalEnv env -> EvalM env a -> Either ErrorType a
```

También se agregó una instancia `Alternative`. Esto permite expresar computaciones con respaldo:

```
lookupVar "z" <|> lookupVar "x"
```

Si la primera búsqueda falla, se intenta la segunda en el mismo entorno. Esta operación no cambia el lenguaje de usuario, pero deja una herramienta simple para expresar fallbacks internos manteniendo errores tipados con `ErrorType`.

8 Manejo de errores

Los errores se representan con:

```
data ErrorType
  = DivideByZero
  | UndefinedVariable String
  | DomainError String
```

Esto permite distinguir división por cero, variables no definidas y errores de dominio. El evaluador evita depender de funciones parciales en los caminos principales. Por ejemplo, `log(0)` devuelve un `DomainError` en vez de cortar la ejecución con una excepción.

Para números duales se agregaron funciones seguras:

- `safeLogDual`;
- `safeSqrtDual`;
- `safeAcoshDual`;
- `safeAtanhDual`;
- `safePowDual`.

Estas funciones controlan casos delicados como derivadas no finitas, bases negativas con exponentes no enteros y potencias cuyo resultado sería `NaN` o infinito.

9 Pretty printer

El pretty printer imprime expresiones minimizando paréntesis, pero sin cambiar el significado. Para eso se trabaja con niveles de precedencia y se distingue cuándo una subexpresión debe imprimirse como base, factor o expresión completa.

En nuestro caso se agregaron casos de borde para evitar impresiones ambiguas:

- potencias con base negativa;
- potencias anidadas;
- restas anidadas a derecha;
- divisiones anidadas a derecha.

Además se agregaron tests de roundtrip: se imprime una expresión, se vuelve a parsear y se compara su evaluación. Esto sirve para comprobar que la impresión no altera el árbol semántico.

10 Optimización conservadora

El módulo `Expr` incluye una función `optimize` con reglas algebraicas simples:

```
x + 0 = x
x * 1 = x
x / 1 = x
x^1 = x
2 + 3 = 5
2^3 = 8
```

La optimización es conservadora. No aplica reglas como:

```
x / x = 1
x * 0 = 0
x^0 = 1
1^x = 1
```

Aunque esas identidades pueden ser válidas bajo ciertas hipótesis, en el evaluador podrían ocultar errores. Por ejemplo, si $x = 0$, entonces x/x debe seguir dando división por cero y x^0 debe seguir respetando el caso 0^0 .

11 Procesamiento de archivos

El modo archivo usa líneas del tipo:

```
expresion @ valor
```

Por ejemplo:

```
sin(x) @ pi/2
x^2 + 2*x + 1 @ 3
```

El valor a la derecha de `@` puede ser un número o una expresión constante. También se soportan comentarios de línea y comentarios de bloque, lo que facilita escribir archivos de prueba más claros.

12 Comparación con un CAS simbólico

Un sistema de álgebra computacional suele apuntar a un alcance más amplio, con derivación simbólica, integración, simplificaciones avanzadas y manipulación algebraica general.

Este proyecto toma una dirección más acotada:

- se enfoca en diferenciación automática;
- evita convertirse en un CAS completo;
- usa números duales como técnica central;
- usa mónadas para ordenar errores y entorno.

Esta diferencia de alcance es importante. Implementar un CAS completo haría crecer mucho el trabajo y desviaría el eje original presentado, que estaba centrado en números duales y diferenciación automática.

13 Tests y validación

La suite de tests cubre:

- evaluación básica;
- evaluación dual;
- parser;
- funciones trigonométricas e hiperbólicas;
- constantes;
- errores de dominio;
- potencias con bases negativas;
- optimizaciones conservadoras;
- pretty printer;
- uso de `EvalM`;
- procesamiento de valores constantes y comentarios en archivos.

La validación actual incluye:

```
cabal check
cabal build all
cabal test
cabal exec -- runghc -isrc test\TestSuite.hs
cabal sdist
```

La suite actual contiene 125 casos, sin errores ni fallos.

14 Limitaciones

Las principales limitaciones actuales son:

- no hay derivación simbólica general;
- no hay integración;
- las simplificaciones son básicas;
- el modo interactivo y el modo archivo siguen centrados en la variable x ;
- el soporte multivariable existe parcialmente en la API, no como lenguaje completo de usuario.

15 Trabajo futuro

Posibles extensiones:

- permitir elegir variable de derivación desde la entrada de usuario;
- extender el formato de archivo para entornos multivariables;
- mejorar mensajes de error;
- agregar tests automáticos para todos los archivos de ejemplo;
- sumar más funciones matemáticas si el alcance lo justifica.

16 Conclusión

El proyecto muestra cómo representar y evaluar un lenguaje pequeño de expresiones matemáticas en Haskell. La diferenciación automática con números duales permite calcular valor y derivada en una sola pasada. La incorporación de `EvalM` ordena el manejo de errores y del entorno, y conecta el trabajo con el estudio de mónadas de la materia.

El alcance final es acotado pero defendible: no intenta resolver álgebra simbólica completa, sino presentar un DSL/EDSL chico, testeado, robusto y con decisiones de diseño claras.